

使用的 AI 模型

主要使用 DeepSeek 对话式 AI 助手，用于技术问答、命令生成、错误日志分析和概念解释。在部分环节也参考了 NVIDIA 官方文档、SGLang GitHub Issues 和 Stack Overflow 的社区回答进行交叉验证。

使用的 AI 提示词

提示词包括：“在 WSL 的 Ubuntu 中安装 Conda” “SGLang 0.5.14 支持哪些 CUDA 版本” “如何查看 WSL 分配的空间” “sgl_kernel 报错 undefined symbol 怎么解决” “CUDA 13.0 和 glibc 的 rsqrt 声明冲突如何修补” “Mooncake trace 采样 workload 的 Python 脚本怎么写” “用一段话解释 PagedAttention 和 RadixAttention 的联系与差异”

AI 如何帮助学习AI

在以下方面提供了显著帮助：概念解释：用通俗语言讲清 RadixAttention、PagedAttention、page-sized KV cache、prefix reuse 等机制，并对比 vLLM 与 SGLang 的设计差异。错误定位：面对 undefined symbol、rsqrt 编译错误、401 Unauthorized 等报错，AI 帮助分析根因，区分是 ABI 不匹配、架构不匹配还是网络问题。命令与脚本生成：提供 Conda 环境创建、CUDA 头文件修补、Mooncake trace 采样脚本等可直接运行的代码。版本兼容性梳理：帮助理清 Windows 驱动、CUDA、PyTorch、SGLang、sgl_kernel 之间的版本依赖关系。在哪些方面使用 AI主要在 环境搭建与依赖排查（WSL、CUDA、PyTorch、SGLang 的安装与冲突解决）、概念理解（推理框架的调度、缓存、内存管理机制）、实验脚本编写（Mooncake trace 采样与请求发送）以及 错误日志分析 时使用 AI。

运用 AI 时是否被误导有被误导的情况。

例如：AI 曾给出错误的 CUDA_HOME 路径（写成 ../torch/nvidia/cuda_runtime，实际应为 ../nvidia/cuda_runtime），后来通过 find 命令自行纠正。AI 建议安装的某个 sgl_kernel 版本在 GitHub Release 中不存在（404），需要根据实际可用版本调整。AI 对 SGLang 0.5.14 的 CUDA 支持描述不够精确，初期误以为可直接用 CUDA 12.8，实际官方默认 CUDA 13，最终通过升级驱动和 Toolkit 解决。